Les filtres dans React (et JavaScript) et reduce

Les filtres dans React (et JavaScript)

1. Définition

Un **filtre** est une **opération qui permet de sélectionner uniquement certains éléments d'une liste** selon un ou plusieurs **critères**.

En d'autres termes :

Le filtre **ne modifie pas la liste originale**, il **crée une nouvelle liste** contenant uniquement les éléments qui respectent la condition.

2. Exemple simple en JavaScript

Imaginons une liste de produits :

```
const produits = [
  { nom: "T-shirt", prix: 120 },
  { nom: "Jean", prix: 250 },
  { nom: "Chaussures", prix: 400 },
];
```

Si on veut **filtrer les produits à moins de 200 DH** :

```
const produitsFiltres = produits.filter(p => p.prix < 200);
console.log(produitsFiltres);
```

Résultat :

```
[
  { nom: "T-shirt", prix: 120 }
```

]

Explication :

- .filter() est une **méthode native de JavaScript**.
- Elle parcourt chaque élément du tableau produits.
- Elle garde seulement les éléments pour lesquels la **condition est vraie** (p.prix < 200).

3. Pourquoi utiliser les filtres en React ?

Dans une application React, on a souvent :

- une **liste d'éléments** (produits, utilisateurs, commandes…),
- et des **champs de recherche ou sélecteurs** (prix min/max, catégorie, texte…).

Le **filtrage dynamique** permet donc d'afficher :

uniquement les éléments que l'utilisateur souhaite voir,
sans recharger la page.

4. Exemple de logique dans React

1. On stocke tous les produits dans un **state** :
2. const [produits, **setProduits**] = useState([]);
3. On crée un autre state pour la **liste filtrée** :
4. const [**filtered**, **setFiltered**] = useState([]);
5. On filtre selon un critère, par exemple un **prix maximum** :
6. const **filtrerParPrix** = (prixMax) => {
7. const result = produits.**filter**(p => p.prix <= prixMax);
8. **setFiltered**(result);
9. };

10.Puis on affiche **filtered** dans le rendu React :

```
11.{filtered.map(p => (
12.  <div key={p.id}>{p.nom} - {p.prix} DH</div>
13.))}
```

5. Les différents types de filtres possibles

Type de filtre	Exemple de condition	Description
Par prix	p.prix >= min && p.prix <= max	Affiche les produits dans une fourchette de prix
Par nom	p.nom.toLowerCase().includes(searchText)	Recherche un texte dans le nom du produit
Par catégorie	p.categorie === "Homme"	Affiche les produits d'une catégorie
Par promotion	p.promo === true	Affiche les produits en promotion

6. En Résumé

Élément	Rôle
filter()	Fonction JS pour filtrer une liste
includes()	Cherche un mot ou une partie de texte
useState()	Stocker les données et les résultats filtrés
setFiltered()	Met à jour la liste affichée
onChange	Récupère les valeurs du filtre depuis les champs (input/select)

7. La méthode reduce() — Calculs sur les tableaux

1. Définition

reduce() est une méthode JavaScript qui permet de réduire un tableau à une seule valeur.
On l'utilise souvent pour calculer :

- la somme d'un champ,
- la moyenne,
- ou un total général.

2. Exemple simple en JavaScript

Imaginons une liste de nombres :

```
const nombres = [10, 20, 30];
```

On veut calculer la somme de ces nombres :

```
const somme = nombres.reduce((acc, n) => acc + n, 0);
console.log(somme); // Résultat : 60
```

Explication :

- **acc** : l'accumulateur (le total provisoire),
- **n** : l'élément actuel du tableau,
- **0** : la valeur de départ (ici 0).

Le fonctionnement :

Étape	acc	n	Résultat
1	0	10	10
2	10	20	30

Étape	acc	n	Résultat
3	30	30	60

3. Exemple avec des objets

Imaginons une liste de produits :

```
const produits = [
  { nom: "T-shirt", prix: 120 },
  { nom: "Jean", prix: 250 },
  { nom: "Chaussures", prix: 400 },
];
```

On veut calculer la moyenne des prix :

```
const moyenne = produits.reduce((acc, p) => acc + p.prix, 0) / produits.length;
console.log(moyenne); // Résultat : 256.66
```

Ici :

- **p.prix : on additionne le prix de chaque produit.**
- **produits.length : on divise par le nombre total d'éléments.**

4. Application dans React

Dans une application React, on peut utiliser **reduce**() pour :

- calculer la moyenne du prix des produits filtrés,
- calculer le total du stock,
- ou afficher une statistique rapide en bas du tableau.

Exemple :

```
const moyennePrix = produitsFiltres.reduce((acc, p) => acc + p.prix, 0) / produitsFiltres.length;
```

5. En résumé

Élément	Rôle
reduce()	Transforme une liste en une seule valeur (total, moyenne…)
acc	Variable qui accumule les valeurs
p ou n	Élément actuel du tableau
0	Valeur initiale de départ
/ length	Sert à calculer la moyenne

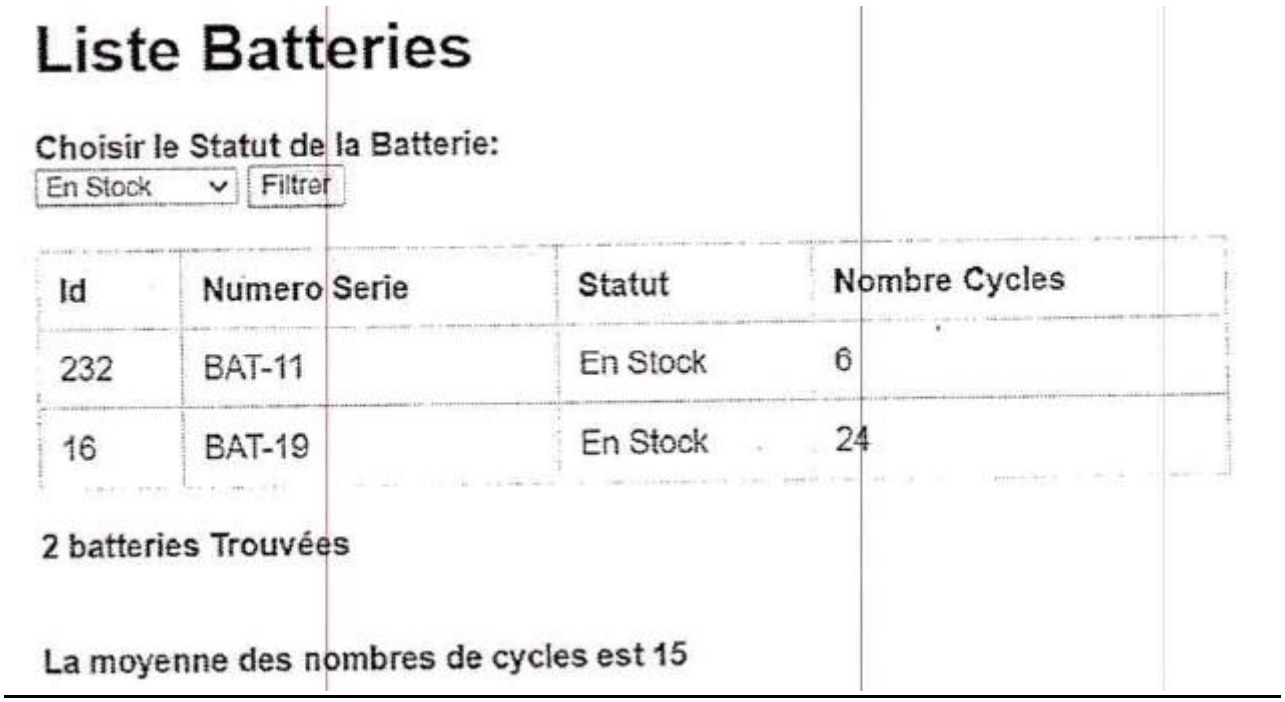
Exemple combiné filter + reduce :

```
const produits = [
  { nom: "T-shirt", prix: 120, statut: "En Stock" },
  { nom: "Jean", prix: 250, statut: "Rupture" },
  { nom: "Chaussures", prix: 400, statut: "En Stock" },
];

const enStock = produits.filter(p => p.statut === "En Stock");
const moyennePrix = enStock.reduce((acc, p) => acc + p.prix, 0) / enStock.length;

console.log("Produits en stock :", enStock);
console.log("Moyenne des prix :", moyennePrix);
```

TP Pratique GUIDE :



Créer une petite application React qui affiche une liste de **batteries** et permet de :

- Filtrer les batteries selon leur **statut** ("En Stock", "Utilisée", "Défectueuse"…).
- Calculer automatiquement :
  - Le **nombre de batteries trouvées**.
  - La **moyenne** du nombre de cycles.

Notions React utilisées :

1. **useState** → pour stocker les données (batteries et filtre choisi).
2. **map()** → pour parcourir le tableau de batteries et afficher chaque ligne du tableau HTML.
3. **filter()** → pour garder seulement les batteries qui correspondent au statut choisi.
4. **reduce()** → pour calculer la moyenne.

5. **onChange** et **onClick** → pour réagir aux actions de l'utilisateur (changer le statut, cliquer sur "Filtrer").

Code complet React avec explications

```
import React, { useState } from "react";

function BatteriesList() {

  // ----------------------------

  //  Données initiales (liste des batteries)

  // ----------------------------

  const batteriesData = [

    { id: 232, numeroSerie: "BAT-11", statut: "En Stock", cycles: 6 },

    { id: 16, numeroSerie: "BAT-19", statut: "En Stock", cycles: 24 },

    { id: 45, numeroSerie: "BAT-05", statut: "Utilisée", cycles: 32 },

    { id: 78, numeroSerie: "BAT-08", statut: "Défectueuse", cycles: 0 },

  ];


  // ----------------------------

  //  useState pour stocker le filtre choisi et les résultats

  // ----------------------------

  const [statutFiltre, setStatutFiltre] = useState("En Stock");

  const [batteriesFiltrees, setBatteriesFiltrees] = useState([]);


  // ----------------------------

  //  Fonction pour filtrer la liste selon le statut
```

```
// ----------------------------
const filtrerBatteries = () => {
  const resultat = batteriesData.filter(
    (batt) => batt.statut === statutFiltre
  );
  setBatteriesFiltrees(resultat);
};

// ----------------------------
//  Calcul du nombre de batteries trouvées
// ----------------------------
const nombreTrouvees = batteriesFiltrees.length;

// ----------------------------
//  Calcul de la moyenne des cycles
// ----------------------------
const moyenneCycles =
  nombreTrouvees > 0
    ? (
        batteriesFiltrees.reduce((somme, b) => somme + b.cycles, 0) /
        nombreTrouvees
      ).toFixed(2)
    : 0;
```

```
// ----------------------------
//  Partie affichage HTML (JSX)
// ----------------------------
return (
  <div style={{ padding: "20px", fontFamily: "Arial" }}>
    <h2>Liste Batteries</h2>

    {/* Zone de filtre */}
    <div>
      <label>Choisir le Statut de la Batterie : </label>
      <select
        value={statutFiltre}
        onChange={(e) => setStatutFiltre(e.target.value)}
      >
        <option>En Stock</option>
        <option>Utilisée</option>
        <option>Défectueuse</option>
      </select>
      <button onClick={filtrerBatteries}>Filtrer</button>
    </div>

    {/* Tableau des résultats */}
    {batteriesFiltrees.length > 0 && (
      <>
```

```
<table border="1" cellPadding="5" style={{ marginTop: "20px" }}>
  <thead>
    <tr>
      <th>Id</th>
      <th>Numéro Série</th>
      <th>Statut</th>
      <th>Nombre Cycles</th>
    </tr>
  </thead>
  <tbody>
    {batteriesFiltrees.map((batt) => (
      <tr key={batt.id}>
        <td>{batt.id}</td>
        <td>{batt.numeroSerie}</td>
        <td>{batt.statut}</td>
        <td>{batt.cycles}</td>
      </tr>
    ))}
  </tbody>
</table>

{/* Résumé */}
<p>
  <strong>{nombreTrouvees}</strong> batteries trouvées
```

```
        </p>
        <p>
          La moyenne des nombres de cycles est <strong>{moyenneCycles}</strong>
        </p>
      </>
    )}
  </div>
 );
}

export default BatteriesList;
```

À tester:

1. Lancer l'application React avec npm start.
2. Choisir différents statuts ("En Stock", "Utilisée"…).
3. Observer le tableau et la moyenne changer automatiquement.